

CYBER-PHYSICAL SYSTEMS PROJECT

Autonomous Plant Keeper

Closing the loop between a living plant and its digital twin

**Group 1 | Mia Sara Christina Dreier, Beyrem Hadj Fredj, Jen-Chien Hou,
Helen Nike Kensbock, Tzuhung Lin, Hussein Nagi, Thanh Tuan Kiet Nguyen**
Embedded Systems, Cyber-Physical Systems and Robotics (INHN0018)

09.09.2026



Agenda

01

Motivation & Objective

Why plants need a digital twin

02

System Design

Requirements, architecture & the feedback loop

03

Hardware & Software

Sensors, actuators, firmware, backend, frontend

04

Video Demo

Watering, lighting & health assessment in action

05

Testing, Limitations & Conclusion

What we verified and what's next

Plants Need Continuous Attention

Light, temperature, humidity and soil moisture all drift over time. Sensitive species, like basil, need near-continuous observation. Manual care only provides intermittent snapshots and fails the caretaker is away.



Intermittent

Manual checks miss changes between visits



Reactive

Care happens after stress is visible



Absence-intolerant

No safety net when no one is around

“Manual monitoring provides only intermittent information and has little tolerance for absence.”

Automated, continuous observation is the natural next step.

The Objective: a Digital Twin for a Plant

Develop a functional prototype that monitors a plant's environment, surfaces that information to a user, and autonomously responds through physical actuators — inspired by industrial predictive maintenance, where a machine's digital twin is monitored and controlled by embedded systems.



Industrial Predictive Maintenance

A complex machine is abstracted into a digital twin, continuously monitored and controlled by embedded systems to predict and prevent failure.



Autonomous Plant Keeper

The same idea applied to a living plant instead of a machine: sense, represent, decide, and act; closing the loop between the physical and cyber worlds.

Requirements & Design Goals

1

Continuous Sensing

Soil moisture, temperature, humidity, pressure & light via dedicated sensors on an ESP32-based controller.

2

Automatic Control

Feedback loop drives grow-light and irrigation actuators from measured conditions — no manual intervention required.

3

Remote Communication

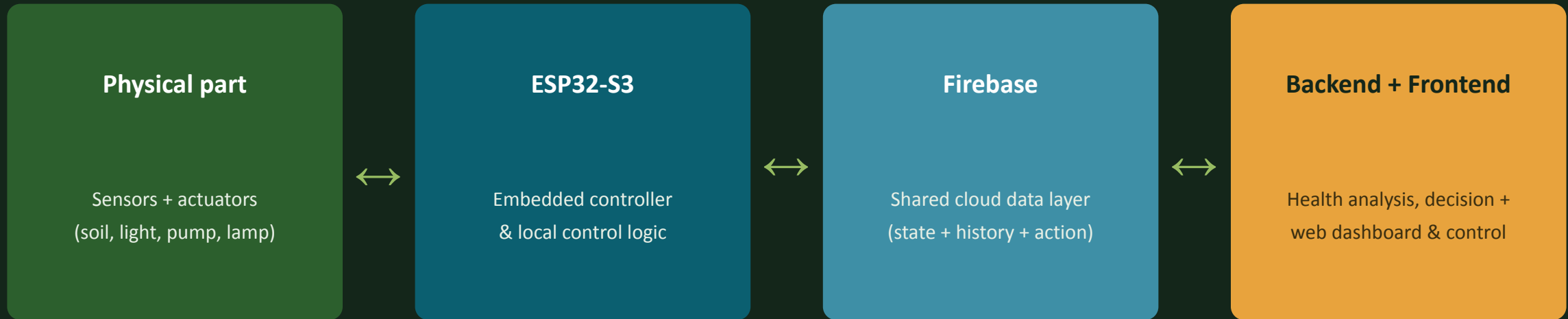
Measurements & states sync to Firebase; a web frontend visualizes data and actuator status from anywhere.

4

Reliable Physical Build

Sensors placed for meaningful readings; electronics secured and separated from water sources.

Four Layers, One Feedback Loop

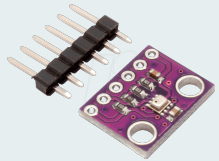


Sensing → embedded processing → shared data → analysis & decision → actuation → back to the physical plant

Components communicate asynchronously through Firebase's shared state — not a single synchronized reaction.

Hardware: the Sensing Subsystem

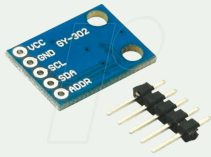
ESP32-S3 DevKitC-1 — GPIO, ADC, I²C and Wi-Fi on one board, acting as the bridge between physical hardware and the cyber layer.



BME280

Temperature, air humidity, pressure

I²C · GPIO 8/9



BH1750

Ambient & effective light intensity

I²C · GPIO 8/9

I²C protocol requires:

- **SDA (Serial Data Line)**: Transmits the data between devices.
- **SCL (Serial Clock Line)**: Carries the clock signal generated by the controller

Hardware: the Sensing Subsystem

ESP32-S3 DevKitC-1 — GPIO, ADC, I²C and Wi-Fi on one board, acting as the bridge between physical hardware and the cyber layer.

- **Soil Sensor:**
 - Produces a voltage that varies with soil moisture -> be read by the ESP32 with ADC (Analog-to-Digital Converter).
 - Calibration defines dry and wet reference values.
- **Water-level Switch:** water lifts a float -> activates the switch -> ESP32 detects the signal change .



Capacitive Soil Sensor

Soil moisture (calibrated %)

Analog · GPIO 1



Water-Level Switch

Reservoir empty/full safety signal

Digital GPIO (ON/OFF) · GPIO 5

Hardware: Actuation & Physical Build

1

Relay-driven actuators

Water pump & grow light on GPIO 2/4, isolated on separate 12V (pump) and 24V (light) supplies from the ESP32 and sensors.

2

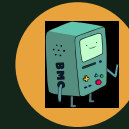
Reservoir safety interlock

The water-level switch hard-blocks pump activation whenever the reservoir is empty.

3

Local feedback

A 2.8 inch LCD display to show sensor values, status, decision and connection state on-device.



BMO-Style 3D-Printed Enclosure

Houses the ESP32-S3 and control electronics, keeping wiring off a breadboard while exposing sensor, actuator, power and display interfaces.

Self-designed wooden frame suspends the grow light above the plant and routes the watering tube — keeping the plant-facing hardware separated from the electronics.

Software: Three Cooperating Layers

1

Embedded Firmware

C · C++

Direct hardware control: sensing, watering fallback, grow-light logic, local display.

2

Backend

Python · Firebase

Random Forest health classification, automatic watering decisions, REST API (realized using Fast API).

3

Frontend

React · Vite

Live dashboard, historical trends, manual controls for the user.

Shared through Firebase Realtime Database: time-critical control stays on-device; heavier analysis runs in the cloud.

Frontend: the Human-Machine Interface

Overview

Moisture, temperature, light, humidity, pressure & VPD at a glance, plus the ML health verdict shown through a BMO-style expression and threshold-based alerts.



Historical Data

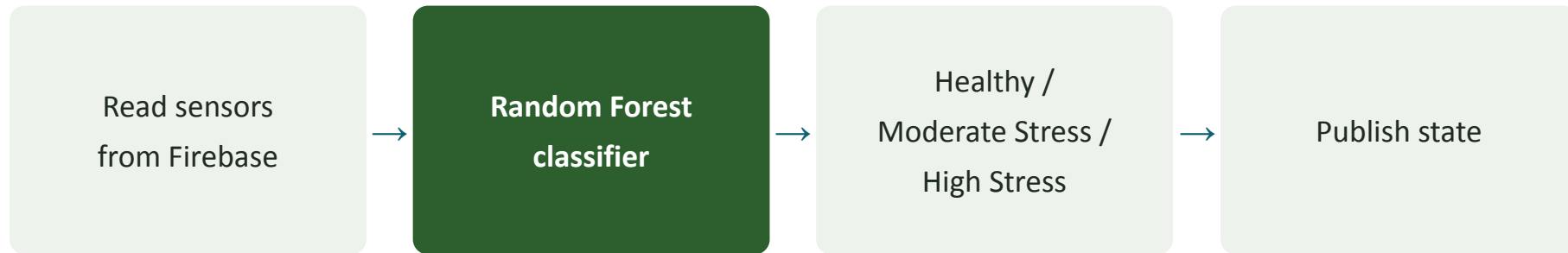
Graphs across selectable ranges, a weighted health score (moisture 40% · temperature 35% · light 25%), and rising/falling/stable trend summaries.



Manual controls: “Water Now” calls the backend; grow-light mode writes straight to Firebase.

Backend: Health Classification & Auto-Watering

Every 30 seconds, the backend loop pulls the latest sensor state, runs it through a trained model, and writes decisions back to Firebase.



Random Forest inputs: soil moisture · ambient temperature · soil temperature · humidity · light intensity

Single source of truth

Watering threshold & pump duration are read directly from the firmware's Config

REST API for the frontend

GET /plant, GET /plant/history and POST /pump expose state and manual control to the dashboard

Embedded Control Logic



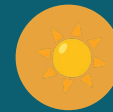
Watering Control

Threshold-based: the pump is triggered once soil moisture drops below threshold.

Backend-driven: normal watering commands arrive via Firebase.

Local fallback: if soil stays dry with no backend response, the ESP32 waters autonomously.

Safety interlock: an empty reservoir always blocks the pump.



Grow-Light Control

Three modes: AUTO, MANUAL_ON, MANUAL_OFF.

Photoperiod: 06:00–22:00 window enforced via synced local time.

Veto: lamp turns off when the room is already bright enough.

Hysteresis + cooldown: prevents relay chatter.

Testing & Evaluation

T1

Environmental Sensing

Sensor values react correctly & propagate to display

T2

Automatic Grow-Light

Lamp responds to ambient light per the auto-control feedback loop

T3

Manual Grow-Light

User can switch modes from the web interface

T4

Automatic Watering

Full sense → decide → actuate → sense loop verified

T5

Manual Watering

“Water Now” triggers the pump on demand

T6

Reservoir Safety

Empty tank correctly blocks pump activation

T7

Monitoring & History

Live and historical views correctly reflect test activity

VIDEO DEMO

See It Water, Light Up & Report Its Own Health



Known Limitations



Limited edge computing

Health classification & the primary watering decision live in the cloud backend, not on-device.



Connectivity dependence

No local database on the ESP32: lost connectivity pauses uploads and cloud-driven commands.



Narrow fault handling

Reservoir checks & fallback watering exist, but not every sensor or actuator failure mode is covered.



Indicative health score

The Random Forest output and dashboard score are indicators, not physiological measurements.

CONCLUSION

A Complete, Closed Cyber-Physical Loop

Sensing, embedded control, cloud communication, ML-based health assessment, physical actuation, and a web interface are integrated into one working prototype, meeting every core requirement defined at the start of the project.

Continuous sensing

Autonomous actuation

Cloud-connected

ML health scoring

Remote dashboard

Local fallback safety

The project demonstrates the core CPS principle: physical information is sensed and processed in the cyber domain, and computational decisions flow back to change the physical world.

Thank You

Questions & discussion welcome

Autonomous Plant Keeper · Cyber-Physical Systems Project

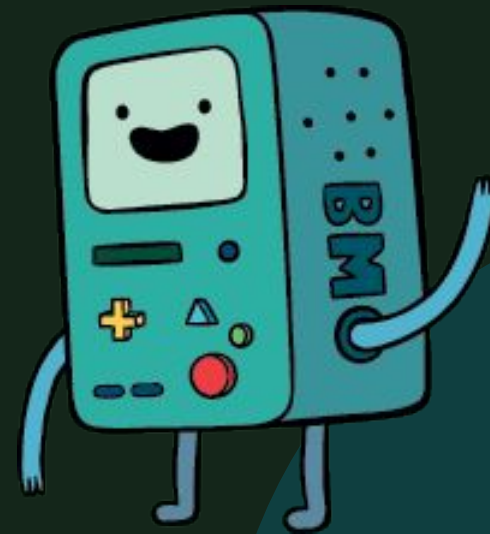


Image Sources



Slide 04

<https://www.veggycation.com.au/vegetables/basil/>



Slide 07

https://www.reichelt.at/at/de/shop/produkt/entwicklerboards_-_temperatur-_feuchtigkeits-_und_drucksensor_-253982
https://www.reichelt.de/de/de/shop/produkt/entwicklerboards_-_digitaler_lichtsensor_bh1750-224217



Slide 08

<https://ca.world.taobao.com/product/%E6%B9%BF%E5%BA%A6%E4%BC%A0%E6%84%9F%E5%99%A8%E6%A8%A1%E5%9D%97.htm>
<https://www.hobbytronics.co.za/p/151/water-level-switch>



Slide 09

<https://adventuretime.fandom.com/wiki/BMO>



Slide 13

<https://www.magnific.com/free-photos-vectors/water-drop-shape>



Slide 18

<https://stickers.gg/sticker/5109-bmo-happy>